

TP3 – Stéréoscopie multivue

Formulation générale du MVS

Si l'on désigne par $\mathbf{P} = [X, Y, Z]^\top$ un point de la surface d'une scène 3D, supposée opaque, et par $\mathbf{p} = [u, v]^\top$ son image formée par une caméra perspective, le lien entre ces deux points s'écrit $\mathbf{p} = \pi(\mathbf{P})$, où π est une projection centrale. Une projection est en général non inversible, mais en supposant que l'on connaisse la *fonction de profondeur* z , définie par $Z = z(u, v)$, il existe bel et bien une bijection entre les points 3D vus par la caméra et leurs images. Nous pouvons dès lors définir la *projection inverse* π_z^{-1} telle que :

$$\mathbf{P} = \pi_z^{-1}(\mathbf{p}) \quad (1)$$

où l'indice z indique que, sans la connaissance de z , une telle écriture n'aurait pas de sens.

Si nous disposons de $n + 1$ images d'une même scène 3D, nous choisissons une de ces images comme *image de référence*, et définissons la fonction de profondeur z , supposée connue, relativement au repère caméra de cette image. Soit $\mathbf{p} = \pi(\mathbf{P})$ l'image d'un point $\mathbf{P}$ de la scène 3D dans l'image de référence. Connaissant les n changements de pose de la caméra correspondant aux n autres images, dites *images témoins*, nous pouvons reprojeter $\mathbf{P}$ dans ces autres images, ce qui donne les n points suivants :

$$\mathbf{p}_k = \pi_k(\mathbf{P}), \quad k \in \{1, \dots, n\} \quad (2)$$

Ces points sont homologues à $\mathbf{p}$, puisqu'ils correspondent tous au même point $\mathbf{P}$ de la scène 3D.

La fonction $I : \mathbb{R}^2 \rightarrow \mathbb{R}^+$ désigne le niveau de gris de l'image de référence (la fonction `rgb2gray` pourra être utilisée, en cas d'images en couleur). Quant au niveau de gris de la $k^{\text{ème}}$ image témoin, il est noté I_k , $k \in \{1, \dots, n\}$. L'*hypothèse lambertienne* $I_k(\mathbf{p}_k) = I(\mathbf{p})$ nous permet d'écrire, en combinant (1) et (2) :

$$I_k \circ \pi_k \circ \pi_z^{-1}(\mathbf{p}) = I(\mathbf{p}), \quad k \in \{1, \dots, n\} \quad (3)$$

Cette égalité devant être vérifiée en tout point $\mathbf{p} = [u, v]^\top$ de l'image de référence, nous pouvons la réécrire :

$$I_k \circ \pi_k \circ \pi_z^{-1}(u, v) = I(u, v), \quad k \in \{1, \dots, n\}, \quad \forall (u, v) \in \Omega \quad (4)$$

où $\Omega \subset \mathbb{R}^2$ désigne l'ensemble des points de l'image de référence qui sont visibles dans toutes les autres images. En pratique, l'égalité (4) n'est jamais parfaitement vérifiée. Dans une démarche de reconstruction 3D, nous pouvons la reformuler comme un problème variationnel d'inconnue z :

$$\min_{z: \Omega \rightarrow \mathbb{R}^+} \sum_{k=1}^n \| I_k \circ \pi_k \circ \pi_z^{-1} - I \|_{\ell^2(\Omega)}^2 \quad (5)$$

où $\| \cdot \|_{\ell^2(\Omega)}^2$ désigne l'intégrale sur Ω de la norme L^2 . Le problème (5) constitue le modèle le plus élémentaire de *stéréoscopie multivue* (en anglais MVS, pour *multi-view stereo*). Notons que, en comparaison des problèmes d'optimisation déjà rencontrés, l'inconnue du problème (5) est *continue*, et non discrète. Nous pouvons donc espérer que cette technique fournisse des reconstructions 3D *denses*, contrairement au SfM. En contrepartie, la résolution risque d'être plus délicate. Commençons par approcher l'intégrale de la norme L^2 dans (5) par une somme discrète, ce qui est d'autant plus justifié que les images numériques sont constituées de pixels $\mathbf{p}$:

$$\min_{z: \Omega \rightarrow \mathbb{R}^+} \sum_{k=1}^n \sum_{\mathbf{p} \in \Omega} \| I_k \circ \pi_k \circ \pi_z^{-1}(\mathbf{p}) - I(\mathbf{p}) \|^2 \quad (6)$$

Dans cette expression, $\mathbf{I}$ est obtenu en vectorisant la matrice de taille $t \times t$ contenant les niveaux de gris des pixels d'un voisinage $t \times t$ centré en $\mathbf{p}$. Il en va de même pour la définition des fonctions $\mathbf{I}_k$, $k \in \{1, \dots, n\}$.

Résolution du MVS

Le problème (6) peut être résolu pixel par pixel, ce qui revient à résoudre, pour chaque pixel $\mathbf{p} \in \Omega$:

$$\min_{z_{i,j} \in \mathbb{R}^+} \sum_{k=1}^n \left\| \mathbf{I}_k \circ \pi_k \circ \pi_{z_{i,j}}^{-1}(\mathbf{p}) - \mathbf{I}(\mathbf{p}) \right\|^2 \quad (7)$$

où $z_{i,j}$ désigne la profondeur $z(\mathbf{p})$ de $\mathbf{p}$, repéré par son numéro de ligne i et son numéro de colonne j .

Vu que l'inconnue $z_{i,j}$ du problème (7) n'apparaît pas de manière directe, mais au travers de la combinaison de fonctions $\mathbf{I}_k \circ \pi_k \circ \pi_{z_{i,j}}^{-1}$, il est difficile d'utiliser les outils de l'optimisation différentiable tels que le gradient ou la hessienne. Il n'est donc plus utile d'élever les résidus au carré. Nous pouvons donc réécrire (7) :

$$\min_{z_{i,j} \in \mathbb{R}^+} \sum_{k=1}^n \left\| \mathbf{I}_k \circ \pi_k \circ \pi_{z_{i,j}}^{-1}(\mathbf{p}) - \mathbf{I}(\mathbf{p}) \right\| \quad (8)$$

Il sera vu dans l'exercice 1 que l'argument du problème (8) évolue de façon très irrégulière en fonction de $z_{i,j}$. C'est la raison pour laquelle il est d'usage de le résoudre en testant N valeurs de $z_{i,j}$ notées $v_1, \dots, v_N$. Une telle méthode de résolution est dite « en force brute » (*brute-force*) :

$$\min_{z_{i,j} \in \{v_1, \dots, v_N\}} \sum_{k=1}^n \left\| \mathbf{I}_k \circ \pi_k \circ \pi_{z_{i,j}}^{-1}(\mathbf{p}) - \mathbf{I}(\mathbf{p}) \right\| \quad (9)$$

Enfin, comme le point $\pi_k \circ \pi_{z_{i,j}}^{-1}(\mathbf{p})$ ne se situe généralement pas au centre d'un pixel de la $k^{\text{ème}}$ image témoin, l'expression $\mathbf{I}_k \circ \pi_k \circ \pi_{z_{i,j}}^{-1}(\mathbf{p})$ doit être calculée par interpolation, par exemple au plus proche voisin (**round**).

Exercice 1 : calcul de l'argument du problème (9)

Le fichier `lapin.mat` contient $n+1 = 5$ images de synthèse $I, I_1, \dots, I_4$ d'un lapin et la matrice de calibrage. Les poses de la caméra sont également fournies, mais **relativement à un repère Monde**. Enfin, la variable `masques` contient les $n+1$ masques du lapin dans les différentes images.

Écrivez la fonction `calcul_argument`, appelée par le script `exercice_1`, qui retourne les n termes de la somme dans le problème (9), pour un pixel $\mathbf{p}$ de l'image de référence sélectionné par l'utilisateur, ainsi que les reprojections de ce pixel dans les n images témoins, pour chaque valeur $v_1, \dots, v_N$ de la profondeur $z_{i,j}$.

Conseils de programmation

- Dans cet exercice, la comparaison des niveaux de gris est effectuée pixel à pixel, c'est-à-dire que $t = 1$ et que $\mathbf{I}$ et $\mathbf{I}_k$ sont des scalaires.
- Après *déprojection* d'un pixel de l'image de référence, et avant *reprojection* dans chacune des images témoins, il est nécessaire de calculer les coordonnées du point 3D dans le repère Monde. Il faut donc effectuer deux changements de repère successifs : $\mathcal{R}_{\text{réf}} \rightarrow \mathcal{R}_{\text{Monde}}$, puis $\mathcal{R}_{\text{Monde}} \rightarrow \mathcal{R}_k$.
- Les indices (i, j) d'un pixel $\mathbf{p}$ (numéro de ligne et numéro de colonne) diffèrent de ses coordonnées $[u, v]^T$ exprimées dans le repère pixels. En effet, non seulement l'ordre est inversé, mais u et v ne sont pas forcément entières, contrairement à i et j . Par conséquent : $i = \text{round}(v)$ et $j = \text{round}(u)$.
- Si la reprojection d'un point 3D dans une image témoin se situe en dehors du masque de cette image, il est conseillé d'affecter la valeur NaN (*not a number*) aux paramètres de sortie correspondants. Attention : avant de tester si la reprojection se situe à l'intérieur du masque ou non, il est nécessaire de vérifier si elle se situe à l'intérieur de l'image témoin, afin d'éviter une erreur à l'exécution.
- Il est recommandé d'utiliser deux boucles `for` imbriquées : une sur les valeurs $v_1, \dots, v_N$ de $z_{i,j}$, l'autre sur les n images témoins.

Exercice 2 : reconstruction 3D par MVS

La comparaison des niveaux de gris pixel à pixel ne peut pas fournir des résultats très précis, surtout pour les images de synthèse d'une surface lisse sans texture, ce qui est le cas des données du fichier `lapin.mat`. La reconstruction 3D sera forcément meilleure en comparant les niveaux de gris des pixels contenus dans une fenêtre centrée de taille 3×3 . Dorénavant, dans (9), $\mathbf{I}$ et $\mathbf{I}_k$ sont donc des vecteurs de $\mathbb{R}^9$. Pour ce faire, une astuce consiste à concaténer, à chacune des $n + 1$ images du lapin, huit images obtenues par décalages dans les quatre directions cardinales N/E/S/O et les quatre directions intercardinales NO/NE/SE/SO. Cette astuce est réalisée par la fonction `pretraitement`, qui vous est fournie : à partir d'une matrice de taille $l \times c \times n$ contenant n images en niveaux de gris, la fonction `pretraitement` crée une matrice de taille $(l * c) \times 9 \times n$.

Écrivez la fonction `MVS`, appelée par le script `exercice_2`, mettant en œuvre la reconstruction 3D par MVS par résolution du problème (9) pour l'ensemble des pixels $\mathbf{p} \in \Omega$.

Conseils de programmation

- Pour des raisons de temps de calcul, il est hors de question d'écrire la fonction `MVS` en ajoutant à la fonction `calcul_argument` de l'exercice précédent une troisième boucle de parcours des pixels de l'image de référence. Il est préférable de vectoriser les données, à l'aide de la fonction `pretraitement`.
- Le paramètre de sortie `erreurs` de la fonction `MVS` doit être une matrice bidimensionnelle, de nombre de lignes égal au nombre de pixels de l'image de référence situés à l'intérieur du masque, et de nombre de colonnes égal au nombre N de valeurs de la profondeur testées.
- Durant la phase de mise au point, les images et les masques sont redimensionnés à l'échelle 0,25 (fonction `imresize` de Matlab) afin d'accélérer les calculs. Observez dans le script `exercice_2` que la matrice $\mathbf{K}$ est modifiée elle aussi, à l'exception de l'élément $\mathbf{K}(3,3)$ qui doit rester égal à 1. Une fois le code mis au point, vous pourrez affecter la valeur 1 à la variable `echelle` (cf. figure 1).

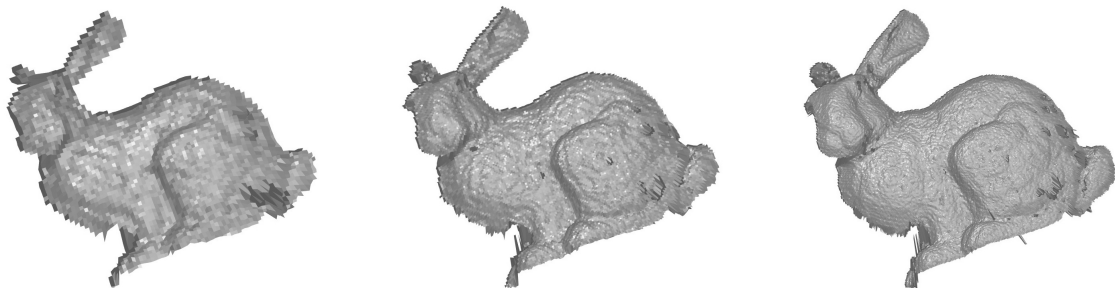


FIGURE 1 – Résultats du MVS pour trois échelles de redimensionnement des images (0,25, 0,5 et 1).

Validation sur les images réelles de la fontaine

Une fois l'exercice 2 mis au point, lancez le script `fontaine` qui permet de reconstruire la fontaine des deux premiers TP. Le fichier de données `fontaine.mat` contient : $n + 1 = 11$ images réelles de la fontaine ; les masques associés à ces images, qui « détournent » la fontaine (le sol et les éléments du fond ont été escamotés) ; la matrice de calibrage ; les poses de la caméra, relativement au repère Monde.